

DISCOVERY PHASE – COMPLETE (Zero assumptions; requirements locked verbatim from your query + full Sovereign Machina architecture + changelog v3.2.7 + latest repo state)

Core functionality required:

- Decide between two paths for the Droid Computer concept in Sovereign Machina:
 1. **Keep it half-Droid oriented** (leverage existing Droid CLI active on Node B, integrate as a specialized execution runtime).
 2. **Completely rebuild as “Hermes Computer”** (native Rust/Go runtime built on top of Hermes supervisor, from scratch).
- Focus on the current tri/quad-node stack as a private, self-improving personal intelligence assistant.
- Maintain: Node A = Mooncake KV buffer, Node B = Director, Node C = Oracle / Voice Core with Omi + Hermes, optional Hermes Pod. No changes to Node A or Node B core logic unless explicitly recommended. Zero-trust, physical sovereignty, local-first design.

All requirements are now exhaustive and production-locked.

RESEARCH PHASE – RIGOROUS CROSS-CHECK (Droid Computer + Sovereign Machina alignment)

Droid Computers (from FactoryAI) are persistent, stateful agent runtimes with isolated filesystem, credentials, resumable sessions, and remote orchestration. Your existing Droid CLI on Node B already gives you a working entry point. Hermes is your native metacognitive supervisor (trajectory capture, enrichment, self-healing, GEPA evolution).

RECOMMENDATION

Keep it half-Droid oriented in the short term, then evolve toward a full “Hermes Computer” native runtime.

Why this hybrid path is optimal:

- **Short-term gains:** You already have Droid CLI active on Node B. Use it immediately as a **specialized execution runtime** for background tasks, persistent sessions, and remote orchestration. This gives you quick value without rebuilding everything from scratch.

- **Long-term sovereignty:** Gradually replace Droid-specific parts with a native Rust runtime built on top of Hermes (the “Hermes Computer”). This keeps full control, avoids external dependencies, and leverages your existing Rust sidecars and GBrain rewrite plans.
- **Workload balance:** Droid CLI on Node B handles execution state (filesystem, credentials, resumable jobs). Hermes (on Node C or future Hermes Pod) handles enrichment, auditing, and self-healing. This is cleaner than a full rebuild now.

Detailed Hybrid Implementation Plan

Phase 1: Half-Droid Oriented (Immediate, Low Effort)

- Register one rig (e.g., the Hermes Pod or a spare) as a BYOM Droid Computer.
- Use Droid CLI on Node B for persistent task queuing and session continuity.
- Route voice notes from Machina Terminal → Node C (Omi + Hermes enrichment) → Droid CLI on Node B for execution.
- Keep all data in your Akashik Artery with Ouroboros auditing.
- Benefit: Instant persistent agents and remote orchestration while you refine the native version.

Phase 2: Evolve to Hermes Computer (Native Rust Runtime)

- Build `sovereign-droid-runtime` (Rust crate) on top of Hermes.
- Reuse Droid concepts (persistent filesystem, resumable Minions, session continuity) but implement them natively with your steganography vault, MemPalace graph, and GEPA evolution.
- Once stable, deprecate Droid CLI dependency and run everything under Hermes supervision.
- This gives you a fully sovereign, self-improving “Hermes Computer” that is 100% under your control.

Why Not Full Rebuild Immediately?

- You already have a working Droid CLI on Node B — using it now accelerates testing and delivers value while the native runtime is built.
- A full from-scratch rebuild would delay benefits and risk scope creep. The hybrid path is pragmatic and aligns with your iterative style.

ARCHITECT SIGN-OFF

The optimal path is to keep it **half-Droid oriented in the short term** (leverage your existing Droid CLI on Node B for execution runtime and persistent sessions) while actively building toward a full **Hermes Computer** native runtime in Rust on top of Hermes. This gives you immediate gains in stateful agent execution and remote orchestration without sacrificing long-term sovereignty. The hybrid approach respects your current hardware and stack, allows real-world testing, and positions the native Rust engine as the ultimate self-improving core. Node A and Node B remain untouched in core logic.

Your main sovereign machine remains completely untouched and rock-solid.

Clarifying questions (mandatory before next phase):

1. Do you want the concrete steps to register a rig as a BYOM Droid Computer and route tasks from Hermes?
2. Shall I start the Rust crate skeleton for the native `sovereign-droid-runtime` on top of Hermes?

Reply with your priority and I will deliver the concrete next layer. The Droid/Hermes integration path is ready.